Code Explanation

Code Explanation: AWS Glue PySpark ETL Job Unit TestsThis test suite validates all components of an AWS Glue ETL pipeline that processes customer and order data. Below is a clear breakdown of each section.---

Overview

This file contains comprehensive unit tests for a PySpark-based ETL job that:- Reads customer and order data from S3- Cleans data (removes nulls/duplicates)- Applies SCD Type 2 transformations using Apache Hudi- Calculates customer spending aggregations- Writes results back to S3 and registers tables in AWS Glue Data Catalog---

Test Fixtures (Setup Components)

1. Spark Session Fixture

```
@pytest.fixture(scope="session")def spark():
```

- Creates a local Spark session for testing with 2 cores- Runs once per test session (shared across all tests)- Automatically cleans up after tests complete

2. Sample Data Fixtures

Four fixtures create test datasets:**`sample_customer_data`**: Clean customer records- 4 customers with IDs, names, emails, and regions- No nulls or duplicates**`sample_customer_data_with_nulls`**: Dirty customer data- Contains null email (C002)- Contains "Null" string value (C003)- Contains duplicate record (C001 appears twice)**`sample_order_data`**: Clean order records- 4 orders with IDs, items, prices, quantities, dates, customer IDs- Calculates total spend: price × quantity**`sample_order_data_with_nulls`**: Dirty order data- Contains null price (O002)- Contains "Null" string in date (O003)- Contains duplicate record (O001 appears twice)---

Test Classes

TestConfigManager

Tests configuration parameter loading.**`test_get_job_parameters_with_defaults`**- Verifies default S3 paths and database names are returned- Ensures fallback values exist when config file is missing**`test_get_job_parameters_with_config`**- Mocks file reading to test YAML config parsing- Validates custom parameters override defaults---

TestS3DataReader

Tests reading data from S3.**`test_read_data_safe_csv`**- Mocks S3 read operation- Verifies CSV data is loaded correctly- Checks expected columns exist (custid, name)**`test_read_data_safe_error_handling`**- Simulates S3 read failure- Ensures exceptions are properly raised with error messages---

TestS3DataWriter

Tests writing data to S3.**`test_write_data_safe_parquet`**- Mocks Parquet write operation- Verifies write method is called successfully**`test_write_data_safe_hudi`**- Tests Hudi format writing with required options: - Table name - Record key (orderid) - Precombine field (date)- Verifies Hudi-specific write method is invoked**`test_write_data_safe_error_handling`**- Simulates S3 write failure- Ensures exceptions propagate correctly---

TestDataCleaner

Tests data quality operations.**`test_remove_nulls_and_duplicates_customer`**- Starts with 5 records (including nulls and duplicate)- Removes 3 bad records: 1. Record with null email 2. Record with "Null" string 3. Duplicate record- Verifies 2 clean records remain- Confirms no nulls exist in output**`test_remove_nulls_and_duplicates_order`**- Same logic for order data- Removes records with null prices or "Null" dates- Removes duplicate orders---

TestCatalogManager

Tests AWS Glue Data Catalog integration.**`test_register_table`**- Mocks Spark SQL execution- Verifies table registration commands are issued- Checks at least 2 SQL statements run (CREATE DATABASE, CREATE TABLE)---

TestHudiManager

Tests Apache Hudi SCD Type 2 functionality.**`test_prepare_scd_type2_data`**- Adds SCD Type 2 columns: - `IsActive`: Boolean flag (all set to True initially) - `StartDate`: Record effective start date - `EndDate`: Record expiration date (null for active records) - `OpTs`: Operation timestamp- Verifies all records are marked active**`test_get_hudi_options`**- Tests Hudi configuration generation- Validates required options: - Table name - Record key field - Precombine field (for deduplication) - Table type (COPY_ON_WRITE)---

TestAggregationEngine

Tests business logic calculations.**`test_calculate_customer_aggregate_spend`**- Calculates total spend per customer: `SUM(price × quantity)`- Verifies 3 unique customers in output- Validates specific calculations: - **C001**: (10.50 × 2) + (25.00 × 1) = **46.0** - **C002**: 15.75 × 3 = **47.25**---

TestSparkContextManager

Tests Spark initialization.**`test_initialize_spark_contexts`**- Verifies Spark session, Glue context, and job objects are created- Ensures all required contexts are non-null---

TestEndToEndIntegration

Tests complete ETL workflow.**`test_full_etl_pipeline`**Simulates entire pipeline:1. **Read Phase**: Mock S3 reads for customer and order data2. **Clean Phase**: Remove nulls and duplicates from both datasets3. **Transform Phase**: Apply SCD Type 2 columns to orders4. **Aggregate Phase**: Calculate customer spending totals5. **Write Phase**: Mock Parquet write to S36. **Catalog Phase**: Mock table registration in Glue Data Catalog**Assertions at each step**:- Data counts match expectations- Required columns exist- Write and catalog operations are invoked---

Key Testing Patterns Used

1. **Mocking**: Uses `unittest.mock` to avoid actual S3/Glue operations2. **Fixtures**: Reusable test data across multiple tests3. **Isolation**: Each test is independent and doesn't affect others4. **Coverage**: Tests happy paths, error handling, and edge cases5. **Integration**: End-to-end test validates component interactions---

Running the Tests

Execute with:

```
pytest glue_pyspark/src/test/test_job.py -v
```

The `-v` flag provides verbose output showing each test result.